

How to use the GOAP AI Tool

This document is a practical guide that covers the core mechanics of the tool as well as an example project.

What is GOAP?

GOAP is short for goal oriented action planning which is a Game AI programming pattern based on the STRIPS (Stanford Research Institute Problem Solver) model. That is an AI framework used to create a sequence of actions to reach a specific goal by breaking it down into an abstract representation.

This is achieved by defining an initial state and a goal state which usually consist of several booleans, the so-called world facts. These facts represent the conditions that need to be true to achieve the specific state.

The STRIPS algorithm then tries to achieve the goal state by searching through a list of possible actions to take and chaining them together to a valid action plan.

These actions have preconditions which describe the world state that is needed to execute them. Each action also has effects which define how it changes the world state after performing that certain action.

A simple example of a pseudo GOAP problem

[Initial State | HasFood == false]

[Goal State | HasFood == true]

[Action | GetFood]

[Preconditions | empty | Effects | HasFood = true]

By defining the action, the initial state and the goal, the planner has all the information it needs to start searching for a valid plan that solves this problem.

If the action's effects match the preconditions of the goal state and the initial state matches the preconditions of the GetFood action, the planner can create a valid plan.

Here is an example where the planner can't find a valid solution to achieve the given goal:

[Initial State | HasFood == false, HasMoney == false]

[Goal State | HasFood == true]

[Action | GetFoodFromSupermarket]

[Preconditions | HasMoney == true| Effects | HasFood = true]

In this case, the planner will return null and it will pick another goal.

For more implementation details see: Planner and GOAPTree [TODO Add links.](#)

This pattern creates a comprehensive way for the computer to achieve predefined goals in a dynamic way. The actions themselves are predefined too, but not the order in which they are performed.

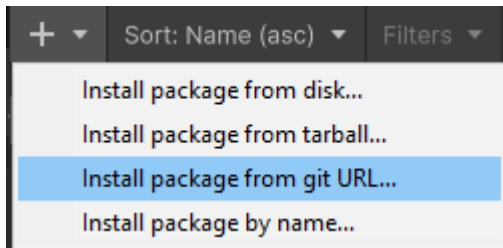
This is the great advantage of GOAP, it allows the user to have a very different workflow compared to using behavior trees, since you don't have to think about the transitioning process and you can focus entirely on the concrete behavior implementation.

Summary:

The purpose of GOAP is to let the computer dynamically create sequences of actions so that the agent can adapt to any situation changes while the game is running instead of using predefined behavior patterns like in state machines or behavior trees.

We start with downloading the package and adding it to your project.

In the Unity editor, open the package manager, click on the plus icon and choose install package by URL:

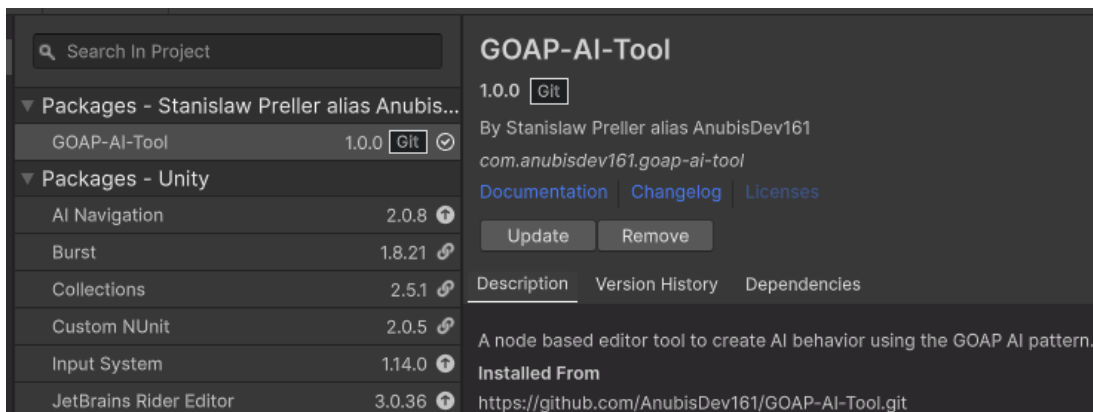


Then enter the package's git URL and hit install:

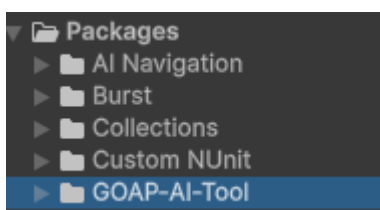


Make sure you wrote .git at the end.

If everything worked, you should now see a new package in your registry:



You should also see the package in your packages folder in your content browser:

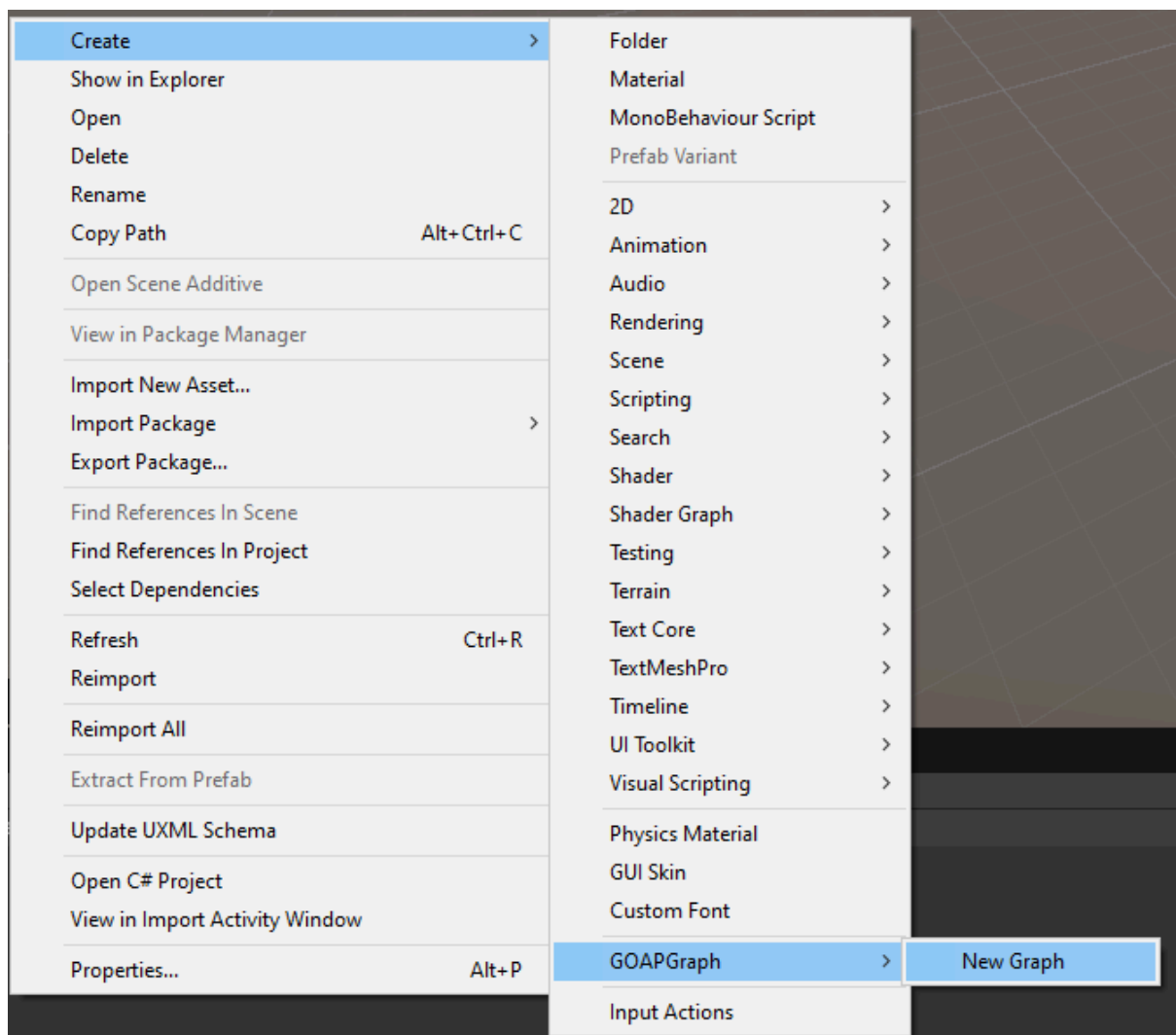


Creating the first behavior

In the asset folder of you Unity project create an empty folder called GOAPGraphs:



Open it and right click in the empty space and go to Create, GOAPGraph, New Graph:

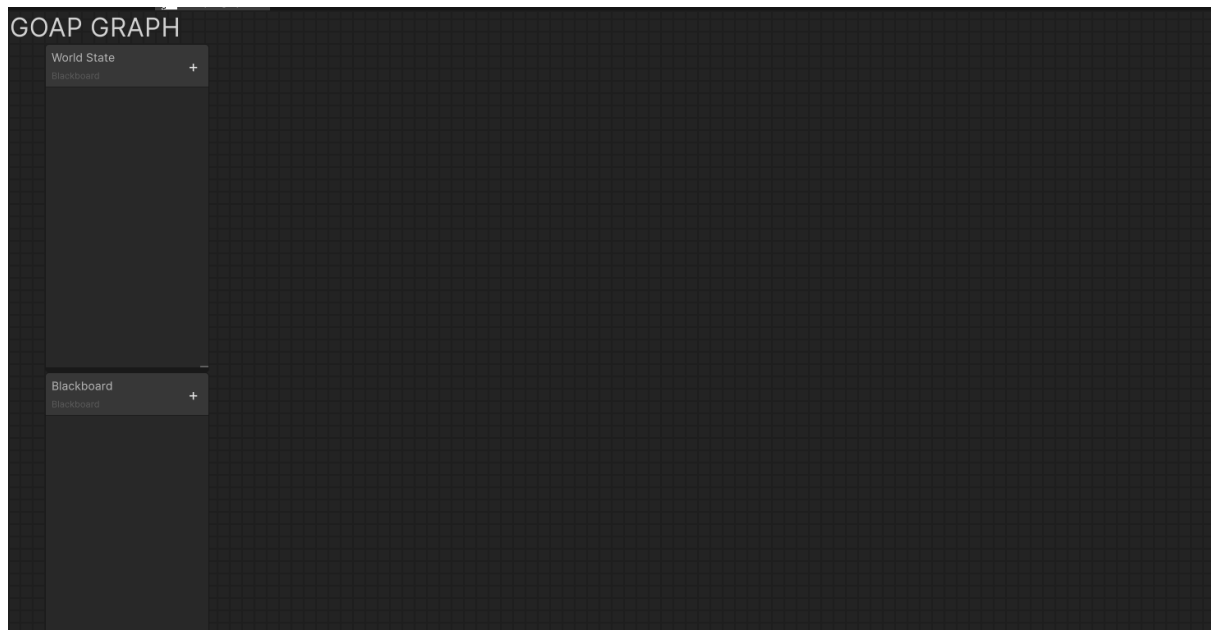


You should now see a new behavior graph asset in your folder:

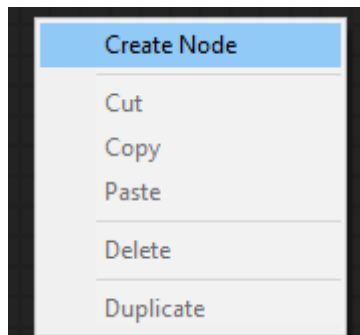


Give it a name and right-click on it.

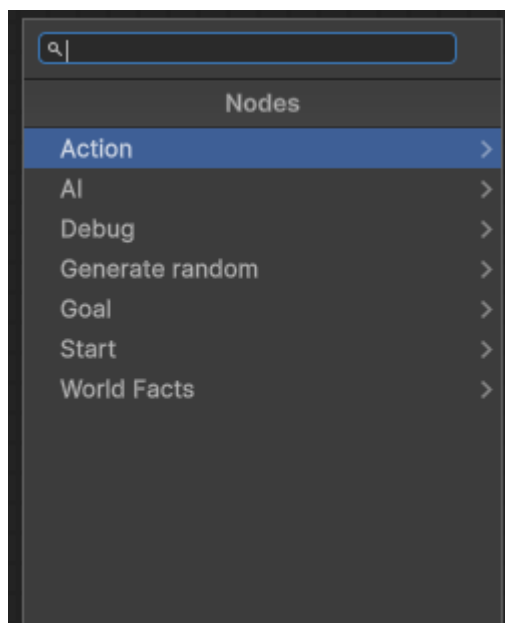
You should now see the GOAP Graph node editor:



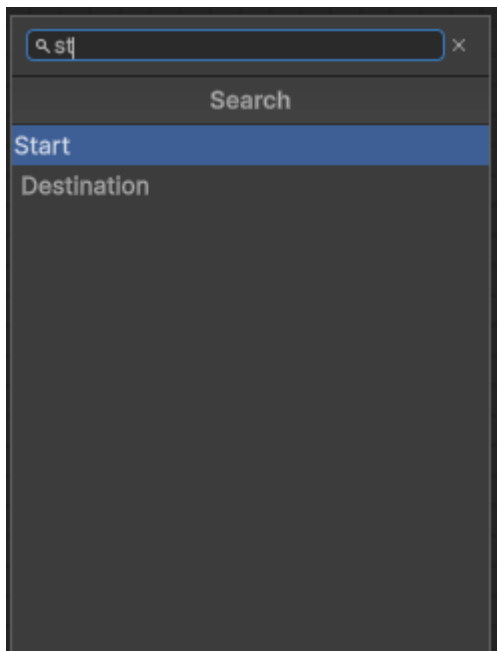
To create behavior, we need to add some nodes to the graph. This is done by right-clicking in empty space of the graph and then clicking Create Node:



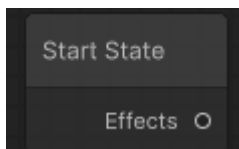
You can now see a search tree:



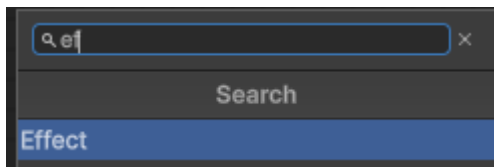
Type start and right click on the found entry:



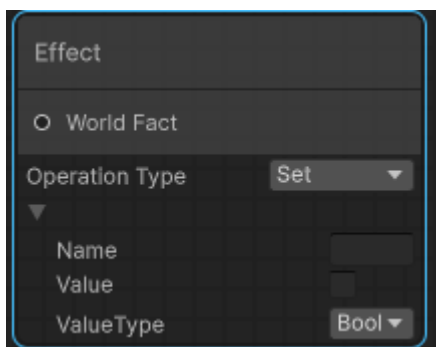
You should now see this node:



Now right click again in the tree, search for Effect and right click on it.



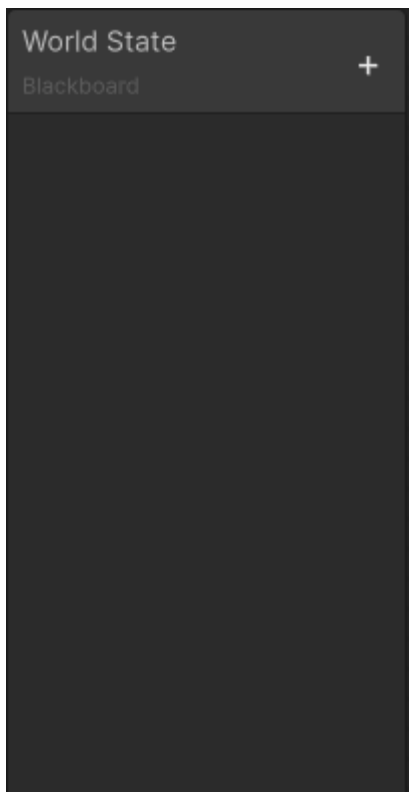
You should now see this node:



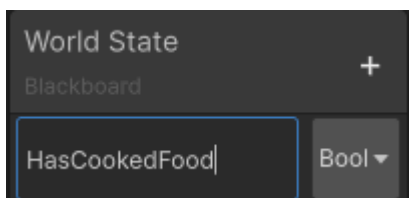
For quick example scenario we want our GOAP agent to cook food, therefore it needs some raw food to cook.

First we define the initial state when the game is starting.

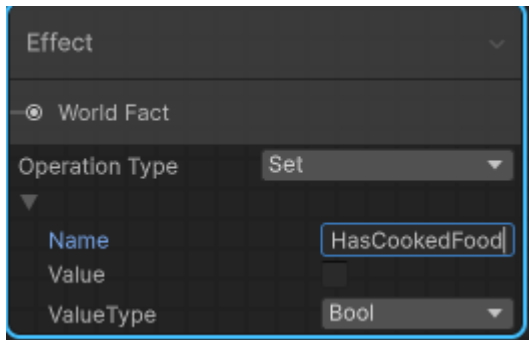
In order to do that, we need a couple of world facts that our actions can use as preconditions or effects. You can define world facts in the world state blackboard on the left side of the editor:



Click on the plus icon and enter a name of our first world fact:

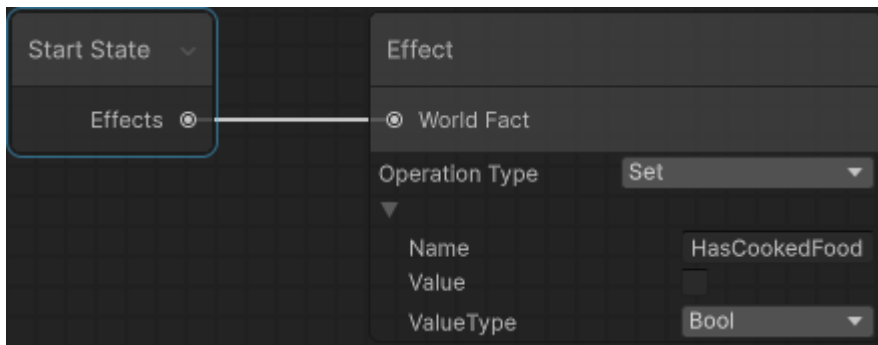


Now you can use this world fact by simply typing its name in the name field of the Effect node:



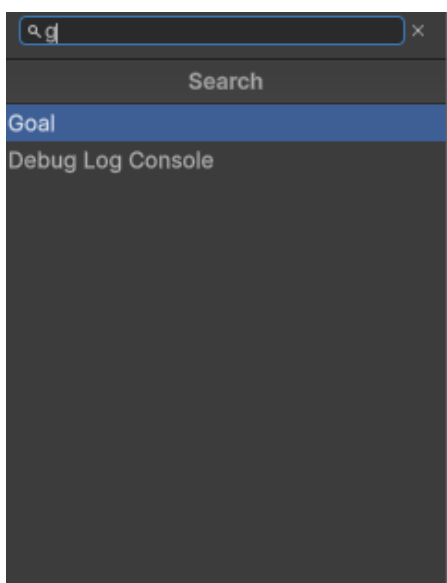
Since the agent has no cooked food in the beginning, you don't need to change the bool value because it is already set to false.

Now connect the Effect node with the start state node:



The initial state is now defined, what is missing is our goal state, which is CookFood.

So right-click again in the empty space of the tree, search for goal and add it to the graph:



Now enter the name of the goal:

The screenshot shows the 'Goal State' configuration panel. At the top, there is a 'Preconditions' section with a radio button. Below it, the 'Name' field is set to 'CookFood'. The 'Priority' is set to '1'. At the bottom, there is a checkbox labeled 'Remove Preconditions From World State After Achieving' which is currently unchecked.

In order to define the goal, you need to add a certain desired world state to it, so add another node: A Precondition node:

The screenshot shows a search interface with a search bar containing the text 'pre'. Below the search bar, there is a 'Search' button and a list of results. The first result, 'Precondition', is highlighted in blue.

Enter the name of the world fact that is supposed to be true:

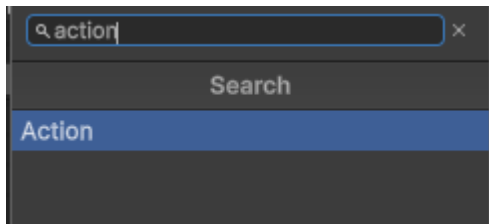
The screenshot shows the 'Precondition' configuration panel. It has a dropdown menu at the top set to 'Precondition'. Below it, there is a 'World Fact' section with a radio button. The 'Accepted Value' is set to 'Equals'. The 'Name' field is set to 'HasCookedFood'. The 'Value' field is empty. The 'ValueType' is set to 'Bool'.

Now connect the node with the goal state node:

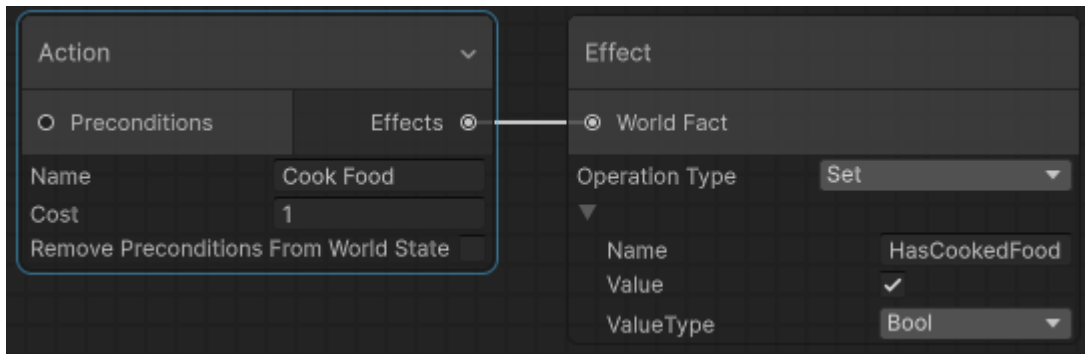
The screenshot shows two panels side-by-side. The left panel is the 'Precondition' configuration panel, and the right panel is the 'Goal State' configuration panel. A line connects the 'World Fact' radio button in the 'Precondition' panel to the 'Preconditions' radio button in the 'Goal State' panel. The 'Preconditions' radio button in the 'Goal State' panel is now selected.

We have now defined our initial state as well as our goal state. The last thing we need to add is an action that has the effects that the goal requires.

Search for action and add it:



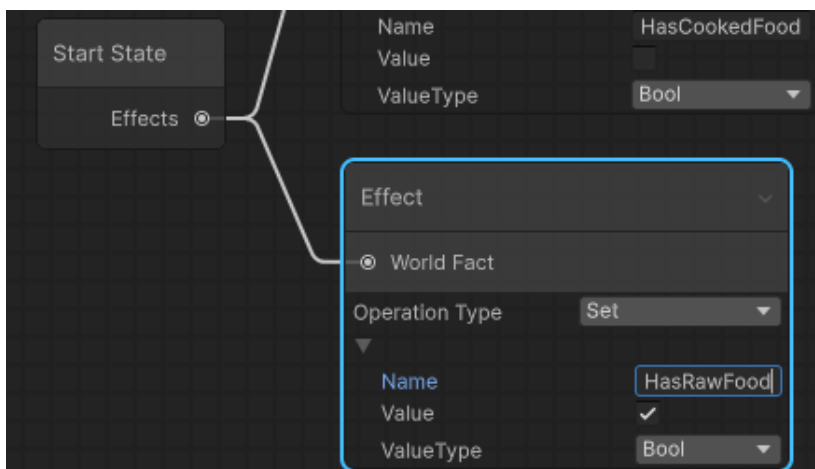
Call it Cook Food and add an effect node with the world fact HasCookedFood that is set to true:



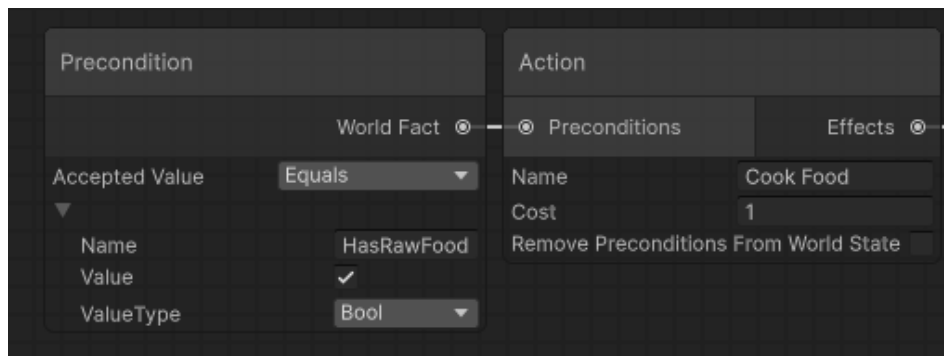
In order to cook food, you need raw food, which is the second and last world fact we need to add in this scenario:



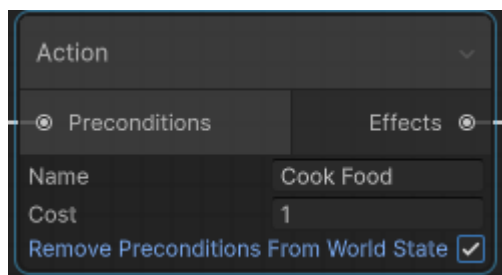
Add another effect node to the Start State which is HasRawFood equals true:



The last thing we need to add in the behavior graph is a precondition node for the Cook Food action that makes sure that it can only cook food, if it has raw food:



Don't forget to hit the Remove preconditions from world state check box, to ensure, that the raw food is being removed after cooking it:



We have now created all the required behavior that we need for this scenario.
Now we have to actually run the tree.

To do that, create an empty folder with the name GameObjects and create a new MonoBehaviour script called ExampleAgent.

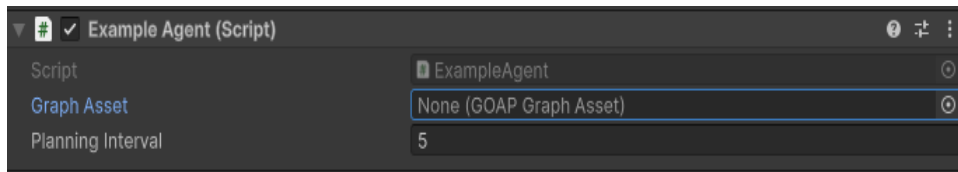
```

Unity Script | 0 references
public class ExampleAgent : GOAPAgent
{
    ...
}

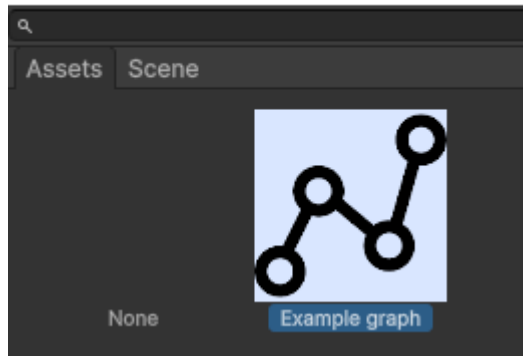
```

Let it inherit from GOAPAgent and close the script.

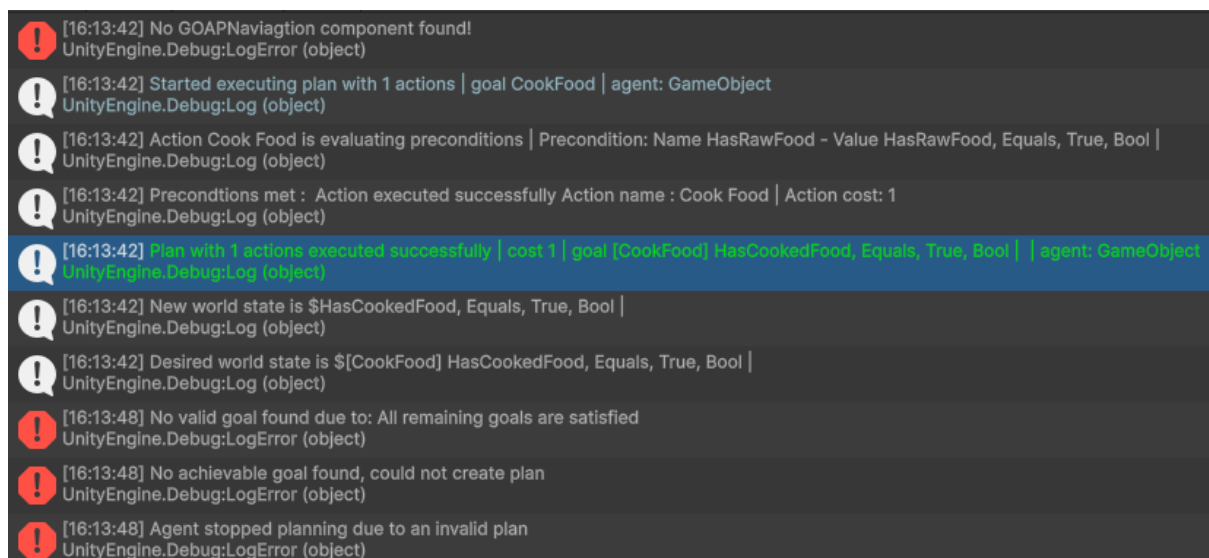
Now add an empty GameObject to the scene and attach the ExampleAgent script to it. Click on the graph asset reference and select the Example Graph:



Select GOAP Graph Asset



The last thing you have to do is to hit play. If you followed this guide correctly, you should now see these prompts in the unity console:



The first error can be ignored since we don't need navigation in this scenario.

Below this error you can see that a valid plan was found and the action was executed successfully.

After 5 seconds the agent tries to plan again and will try to find a valid goal to pursue. But it will then stop planning because the CookFood goal is still achieved.

Now that we have covered the basics of creating goals and actions you can start making your own logic using built-in actions like MoveTo or generate randomDestiantion.

The only difference between the basic action and those two nodes is that they contain gameplay logic which can be created easily by letting a custom action node derive from actionNode:

With just a few lines of Code you can instantly write gameplay code that is called when the action gets executed, in order to do that you just have to override the OnExecute method of the action node.

Here is an example of the “chop tree” action node:

```
[NodeInfo("Chop tree", "Example / Chop tree", hasInputParams: true, hasOutputParams: true)]
0 references
public class ChopTree : ActionNode
{
    [ExposedProperty]
    public string woodSourceKeyName;

    6 references
    public override void OnExecute(GOAPGraphAsset currentGraph, WorldState worldState, Dictionary<string, WorldFact> preconditions = null,
    {
        success = false;
        var woodSourceKey = currentGraph.Blackboard.GetKeyWithExpectedType(woodSourceKeyName, GOAPBlackbaord.BlackboardKeyType.GameObject);
        var woodSource = woodSourceKey.value as GameObject;
        if (woodSource != null)
        {
            success = true;
            GameObject.Destroy(woodSource);
        }

        base.OnExecuteFinished(currentGraph, worldState, success);
    }
}
```